```
import numpy as np

class TrafficMDP:
    def __init__(self):
        self.states = ['LOW', 'MEDIUM', 'HIGH']
        self.actions = ['SHORT', 'MEDIUM', 'LONG']

        # Transition probabilities: P(next_state | state, action)
        self.transitions = {
            'LOW': {
                'SHORT': [('LOW', 0.7), ('MEDIUM', 0.3)],
                'MEDIUM': [('LOW', 0.8), ('MEDIUM', 0.2)],
                'LONG': [('LOW', 0.9), ('MEDIUM', 0.1)]
            },
            'MEDIUM': {
                'SHORT': [('MEDIUM', 0.6), ('HIGH', 0.4)],
                'MEDIUM': [('LOW', 0.3), ('MEDIUM', 0.5), ('HIGH', 0.2)],
                'LONG': [('LOW', 0.5), ('MEDIUM', 0.4), ('HIGH', 0.1)]
            },
            'HIGH': {
                'SHORT': [('HIGH', 0.8), ('MEDIUM', 0.2)],
                'MEDIUM': [('HIGH', 0.6), ('MEDIUM', 0.4)],
                'LONG': [('MEDIUM', 0.6), ('LOW', 0.4)]
            }
        }

        # Rewards (higher is better = less waiting time)
        self.rewards = {
            'LOW': 10,
            'MEDIUM': 5,
            'HIGH': -10
        }


# ---------------- VALUE ITERATION ----------------
def value_iteration(mdp, gamma=0.9, epsilon=0.01):
    V = {s: 0 for s in mdp.states}

    while True:
        delta = 0
        new_V = V.copy()

        for state in mdp.states:
            action_values = []

            for action in mdp.actions:
                value = 0
                for next_state, prob in mdp.transitions[state][action]:
                    reward = mdp.rewards[next_state]
                    value += prob * (reward + gamma * V[next_state])

                action_values.append(value)

            new_V[state] = max(action_values)
            delta = max(delta, abs(V[state] - new_V[state]))

        V = new_V

        if delta < epsilon:
            break

    return V
```

```
# ---------------- EXTRACT POLICY ----------------
def extract_policy(mdp, V, gamma=0.9):
    policy = {}

    for state in mdp.states:
        best_action = None
        best_value = float('-inf')

        for action in mdp.actions:
            value = 0
            for next_state, prob in mdp.transitions[state][action]:
                reward = mdp.rewards[next_state]
                value += prob * (reward + gamma * V[next_state])

            if value > best_value:
                best_value = value
                best_action = action

        policy[state] = best_action

    return policy


# ---------------- MAIN ----------------
if __name__ == "__main__":
    mdp = TrafficMDP()

    V = value_iteration(mdp)
    policy = extract_policy(mdp, V)

    print("Optimal State Values:")
    for state, value in V.items():
        print(f"{state}: {round(value, 2)}")

    print("\nOptimal Policy (Signal Timing):")
    for state, action in policy.items():
        print(f"{state} traffic → {action} green time")
```

```
Optimal State Values:
LOW: 90.05
MEDIUM: 84.65
HIGH: 85.12

Optimal Policy (Signal Timing):
LOW traffic → LONG green time
MEDIUM traffic → LONG green time
HIGH traffic → LONG green time
```